Second Interim Report:
Robot Arm Simulation with Inverse Kinematics and 3D Projection

Roman Prokhorov, Nazar Pasichnyk, Mykola Balyk

April 2026

Abstract

This report presents the progress and theoretical foundations of our Robot Arm Inverse Kinematics Visualization project. The objective is to develop an interactive 3D simulator that demonstrates how linear algebra techniques—specifically matrix transformations, the Jacobian matrix, and Singular Value Decomposition—enable a robotic arm to reach user-specified target positions in real time. This document extends the first interim report with detailed mathematical foundations, a comprehensive literature review, and an updated implementation pipeline.

1 Introduction

Robotic manipulation is a central challenge in robotics, computer animation, and industrial automation. At the heart of this field lies the **inverse kinematics** (IK) problem: given a desired position of a robot's end-effector (e.g., a gripper or tool), determine the joint angles required to achieve that position [1]. Unlike the straightforward *forward kinematics*—where joint angles directly yield the end-effector position via sequential matrix multiplication—inverse kinematics requires solving a nonlinear system that may have zero, one, or infinitely many solutions.

The research goal of this project is to investigate whether linear algebra methods—specifically Jacobian-based iterative solvers with SVD-regularized pseudoinverses—can provide a robust, generalizable approach to real-time inverse kinematics for articulated robot arms, and to demonstrate these mathematical concepts through an interactive visualization.

This problem is deeply rooted in **linear algebra**. The kinematic chain of a robot arm is modeled as a product of 4×4 homogeneous transformation matrices (combining rotations from the Special Orthogonal Group SO(3) with translations). The iterative IK solution relies on the **Jacobian matrix**—a linear approximation of how joint angle changes affect end-effector position—and its **pseudoinverse** computed via Singular Value Decomposition for numerical stability. Visualizing the arm in real time additionally requires perspective projection matrices and orthonormal basis construction for camera transformations.

The significance of this project lies in connecting abstract algebraic concepts directly to a tangible, visual application: a moving robot arm that tracks a target in 3D space, where every frame rendered and every joint angle solved is a direct application of core Linear Algebra concepts.

2 Literature Review and Related Work

2.1 Approaches to Inverse Kinematics

Several complete approaches exist for solving inverse kinematics, each with distinct characteristics [1, 3]:

1. **Analytical (Closed-Form) Solutions**: For specific arm geometries with known structure (e.g., 6-DOF arms with spherical wrists), closed-form solutions can be derived algebraically. These are computationally fast but limited to particular kinematic configurations and cannot be generalized [3].

2. **Jacobian Transpose Method**: Updates angles via $\Delta\theta = \alpha J^T \Delta p$, where α is a step size. This method is computationally inexpensive but exhibits slow convergence and sensitivity to step size selection.
3. **Jacobian Pseudoinverse Method**: Uses $\Delta\theta = J^+ \Delta p$ where $J^+ = J^T(JJ^T)^{-1}$ is the Moore-Penrose pseudoinverse. This provides faster convergence and a minimum-norm solution but becomes numerically unstable near kinematic singularities where the Jacobian loses rank.
4. **Damped Least Squares (DLS)**: Modifies the pseudoinverse as $\Delta\theta = J^T(JJ^T + \lambda^2 I)^{-1}\Delta p$, adding a damping factor λ to prevent numerical blow-up at singularities [1]. This is mathematically equivalent to Tikhonov regularization.
5. **Cyclic Coordinate Descent (CCD)**: A heuristic method that adjusts one joint at a time to minimize the end-effector error. While easy to implement, it can produce unnatural, jerky motion.
6. **FABRIK (Forward And Backward Reaching IK)**: A geometric method operating on positions rather than angles. Fast and intuitive but less suited for demonstrating linear algebra concepts.

2.2 Chosen Method: Damped Least Squares with SVD

After evaluating these approaches, the authors selected the **Damped Least Squares** method with **SVD-based pseudoinverse** computation. This choice was made because:

Advantages:

- Directly demonstrates key linear algebra concepts (Jacobian, SVD, matrix decomposition, pseudoinverse)
- Handles singularities gracefully through damping and singular value thresholding
- Converges reliably for reachable targets with predictable behavior
- Generalizes to arbitrary joint configurations without geometric assumptions

Disadvantages:

- Computationally more expensive than transpose method (SVD per iteration)
- Damping parameter λ requires tuning for optimal performance
- Local optimizer—may converge to suboptimal configurations
- Does not guarantee reaching targets outside the workspace

3 Theoretical Foundation: Linear Algebra Methods

3.1 Homogeneous Coordinates and Transformation Matrices

To unify rotation and translation into a single matrix operation, the authors employ **homogeneous coordinates**. A 3D point (x, y, z) is represented as a 4D vector $\mathbf{p} = [x, y, z, 1]^T$. This enables combined rotation and translation through a single 4×4 matrix multiplication:

$$\mathbf{p}' = T \cdot R \cdot \mathbf{p} = \begin{bmatrix} R_{3\times 3} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \mathbf{p}$$

where $R \in SO(3)$ is a rotation matrix and $\mathbf{t} \in \mathbb{R}^3$ is the translation vector.

3.2 Rotation Matrices and the Special Orthogonal Group SO(3)

Rotation around each principal axis is represented by a matrix in the **Special Orthogonal Group SO(3)**—the group of 3×3 orthogonal matrices with determinant $+1$:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}, \; R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \; R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

Key properties exploited in the implementation:

- **Orthogonality**: $R^{-1} = R^T$ (inverse equals transpose)
- **Determinant**: $\det(R) = 1$ (preserves orientation and volume)
- **Composition**: $R_{total} = R_1 R_2 \cdots R_n$ (non-commutative multiplication)

3.3 Forward Kinematics via Matrix Chain Multiplication

For a robot arm with n joints, each joint i has a joint angle θ_i, a local rotation matrix $R_i(\theta_i)$, and a link length l_i. The transformation matrix for joint i in homogeneous form:

$$T_i(\theta_i) = \begin{bmatrix} R_i(\theta_i) & \mathbf{t}_i \\ \mathbf{0}^T & 1 \end{bmatrix}$$

The **end-effector position** is obtained by chaining all joint transforms:

$$T_{end} = T_1(\theta_1) \cdot T_2(\theta_2) \cdot \ldots \cdot T_n(\theta_n)$$

The position is extracted as $\mathbf{p}_{end} = T_{end} \cdot [0, 0, 0, 1]^T$. This is pure matrix multiplication—the entire kinematic chain is a product of 4×4 matrices.

3.4 The Jacobian Matrix

The **Jacobian matrix** J relates infinitesimal joint angle changes to end-effector velocity:

$$\Delta \mathbf{p} \approx J(\boldsymbol{\theta}) \cdot \Delta \boldsymbol{\theta}$$

where J is a $3 \times n$ matrix (3 spatial dimensions, n joints):

$$J = \begin{bmatrix} \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} & \cdots & \frac{\partial p_x}{\partial \theta_n} \\ \frac{\partial p_y}{\partial \theta_1} & \frac{\partial p_y}{\partial \theta_2} & \cdots & \frac{\partial p_y}{\partial \theta_n} \\ \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} & \cdots & \frac{\partial p_z}{\partial \theta_n} \end{bmatrix}$$

For revolute joints, each column j is computed using the **cross product**:

$$J_j = \mathbf{a}_j \times (\mathbf{p}_{end} - \mathbf{p}_j)$$

where $\mathbf{a}_j$ is the rotation axis of joint j in world coordinates, and $\mathbf{p}_j$ is the position of joint j.

3.5 Singular Value Decomposition for Stable Pseudoinverse

The Moore-Penrose pseudoinverse $J^+ = J^T(JJ^T)^{-1}$ becomes unstable when JJ^T is near-singular. The authors employ **SVD** to compute a numerically stable pseudoinverse:

$$J = U \Sigma V^T$$

where U and V are orthogonal matrices, and $\Sigma = \text{diag}(\sigma_1, \ldots, \sigma_r)$ contains the singular values. The pseudoinverse is:

$$J^+ = V \Sigma^+ U^T$$

For **Damped Least Squares**, the singular values are modified:

$$\sigma_i^+ = \frac{\sigma_i}{\sigma_i^2 + \lambda^2}$$

This damping prevents division by near-zero singular values at kinematic singularities (e.g., fully extended arm configurations), ensuring numerical stability.

3.6 3D Perspective Projection

The rendering pipeline transforms 3D world coordinates to 2D screen coordinates through matrix multiplication and perspective division:

$$\mathbf{p}_{screen} = M_{viewport} \cdot \frac{M_{proj} \cdot M_{view} \cdot \mathbf{p}_{world}}{w}$$

The **perspective projection matrix** creates depth foreshortening:

$$P = \begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{far+near}{near-far} & \frac{2 \cdot far \cdot near}{near-far} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

where $f = \cot(fov_y/2)$.

The **view matrix** is constructed from an orthonormal basis $(\mathbf{r}, \mathbf{u}, \mathbf{f})$ representing camera right, up, and forward directions—computed using cross products and normalization:

$$V = \begin{bmatrix} r_x & r_y & r_z & -\mathbf{r} \cdot \mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $\mathbf{e}$ is the camera position (eye point).

4 Implementation Pipeline

4.1 Software Architecture

The project follows a modular architecture separating mathematical components from visualization:

```
robot_arm/
|-- core/              # Linear algebra primitives
|   |-- vectors.py     # Vector3, Vector4 with dot/cross products
|   |-- transforms.py  # Rotation matrices, homogeneous transforms
|-- kinematics/        # Robotics computations
|   |-- forward.py     # Forward kinematics via matrix chain
|   |-- inverse.py     # DLS IK solver with SVD
|-- rendering/         # 3D graphics
|   |-- projection.py  # Perspective/orthographic projection
|   |-- camera.py      # Orbit camera with view matrix
|-- visualization/     # Application
    |-- app.py         # Interactive Pygame application
```

This modular design ensures each linear algebra concept is isolated in dedicated modules, making the connection between theory and implementation explicit.

4.2 Implemented Components

The system implements all core mathematical and visualization components:

- **Vector operations**: Immutable Vector3/Vector4 classes with dot product, cross product, normalization, and homogeneous coordinate conversion
- **Transformation matrices**: Rotation matrices R_x, R_y, R_z from SO(3), homogeneous 4×4 transforms, matrix composition and inversion

- **Forward kinematics**: Multi-joint robot arm model with matrix chain multiplication for end-effector position
- **Inverse kinematics**: Damped Least Squares solver with SVD-based pseudoinverse computation
- **3D projection**: Perspective and orthographic projection matrices, viewport transformation
- **Camera system**: Orbit camera with spherical coordinate control, view matrix from orthonormal basis
- **Interactive visualization**: Real-time Pygame application with mouse/keyboard controls
- **Testing framework**: Comprehensive pytest suite validating mathematical correctness

4.3 Detailed Algorithm Descriptions

4.3.1 Forward Kinematics

Forward kinematics computes the end-effector position from joint angles via matrix chain multiplication:

Algorithm 1 Forward Kinematics

Require: Joint angles $\boldsymbol{\theta} = [\theta_1, \ldots, \theta_n]$, link lengths $\mathbf{l} = [l_1, \ldots, l_n]$
Ensure: End-effector position $\mathbf{p}_{end}$

```
T ← I_4                                          ▷ Initialize to 4×4 identity
for i = 1 to n do
    R ← RotationMatrix(axis_i, θ_i)              ▷ 3×3 rotation
    t ← [l_i, 0, 0]^T                            ▷ Link offset
    T_i ← [R  t; 0^T  1]                         ▷ Homogeneous form
    T ← T · T_i                                  ▷ Chain multiplication
end for
p_end ← T · [0, 0, 0, 1]^T                       ▷ Extract position
return p_end
```

4.3.2 Jacobian Matrix Computation

The Jacobian relates joint velocities to end-effector velocity using the cross product:

Algorithm 2 Compute Jacobian Matrix

Require: Robot arm with n joints, current configuration $\boldsymbol{\theta}$
Ensure: Jacobian matrix $J \in \mathbb{R}^{3\times n}$

```
T_frames ← ComputeJointTransforms(θ)             ▷ FK for all joints
p_end ← T_frames[n].translation
J ← ZeroMatrix(3, n)
for j = 1 to n do
    p_j ← T_frames[j].translation                ▷ Joint position
    a_j ← T_frames[j].rotation · [0, 0, 1]^T     ▷ Rotation axis in world frame
    J[:, j] ← a_j × (p_end − p_j)                ▷ Cross product
end for
return J
```

4.3.3 SVD-Based Damped Pseudoinverse

Computing the damped pseudoinverse via SVD ensures numerical stability:

Algorithm 3 Damped Pseudoinverse via SVD

Require: Jacobian $J \in \mathbb{R}^{3\times n}$, damping factor λ
Ensure: Damped pseudoinverse $J^+ \in \mathbb{R}^{n\times 3}$

```
U, Σ, V^T ← SVD(J)                          ▷ Singular value decomposition
Σ^+ ← ZeroMatrix(n, 3)
for i = 1 to min(3, n) do
    σ_i^+ ← σ_i / (σ_i^2 + λ^2)             ▷ Damped inverse
    Σ^+[i, i] ← σ_i^+
end for
J^+ ← V · Σ^+ · U^T                         ▷ Pseudoinverse
return J^+
```

4.3.4 Complete Inverse Kinematics Solver

The main IK solver iteratively adjusts joint angles to reach the target:

Algorithm 4 Damped Least Squares IK Solver

Require: Robot arm, target $\mathbf{p}_{target}$, damping λ, tolerance ε
Ensure: Joint angles $\boldsymbol{\theta}$

```
θ ← InitialGuess()                          ▷ Start from current or zero
for iteration = 1 to max_iterations do
    p_current ← ForwardKinematics(θ)
    Δp ← p_target − p_current
    if ‖Δp‖ < ε then return SUCCESS, θ
    end if
    J ← ComputeJacobian(θ)
    J^+ ← DampedPseudoinverse(J, λ)
    Δθ ← J^+ · Δp
    θ ← θ + Δθ
    Clamp θ to joint limits
end for
return MAX_ITERATIONS, θ
```

4.3.5 View Matrix Construction

The view matrix transforms world coordinates to camera space using an orthonormal basis:

Algorithm 5 Construct View Matrix

Require: Camera position $\mathbf{e}$, target $\mathbf{c}$, up hint $\mathbf{u}_{hint}$
Ensure: View matrix $V \in \mathbb{R}^{4\times 4}$

$\mathbf{f} \leftarrow \text{normalize}(\mathbf{c} - \mathbf{e})$ ▷ Forward direction

$\mathbf{r} \leftarrow \text{normalize}(\mathbf{f} \times \mathbf{u}_{hint})$ ▷ Right direction

$\mathbf{u} \leftarrow \mathbf{r} \times \mathbf{f}$ ▷ Corrected up (already normalized)

$$V \leftarrow \begin{bmatrix} r_x & r_y & r_z & -\mathbf{r}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

return V

4.3.6 Projection Pipeline

The complete rendering pipeline projects 3D points to 2D screen coordinates:

Algorithm 6 3D to 2D Projection

Require: World point $\mathbf{p}_{world}$, view matrix V, projection matrix P
Ensure: Screen coordinates (x_{screen}, y_{screen})

$\mathbf{p}_{homo} \leftarrow [p_x, p_y, p_z, 1]^T$ ▷ Homogeneous form
$\mathbf{p}_{view} \leftarrow V \cdot \mathbf{p}_{homo}$ ▷ To camera space
$\mathbf{p}_{clip} \leftarrow P \cdot \mathbf{p}_{view}$ ▷ To clip space
$\mathbf{p}_{ndc} \leftarrow [\frac{p_{clip,x}}{p_{clip,w}}, \frac{p_{clip,y}}{p_{clip,w}}, \frac{p_{clip,z}}{p_{clip,w}}]^T$ ▷ Perspective divide
$x_{screen} \leftarrow (p_{ndc,x} + 1) \cdot \frac{width}{2}$ ▷ NDC to pixels
$y_{screen} \leftarrow (1 - p_{ndc,y}) \cdot \frac{height}{2}$ ▷ Flip Y-axis
return (x_{screen}, y_{screen})

4.4 Testing Strategy

The test suite validates mathematical correctness:

- **Rotation matrices**: $R^T R = I$, $\det(R) = 1$
- **FK consistency**: Zero-angle configuration produces expected reach
- **IK convergence**: Error decreases monotonically for reachable targets
- **Singularity robustness**: No NaN at fully extended configurations
- **Projection**: Verified against manually computed test cases

4.5 Testing and Validation

The implementation includes comprehensive testing to validate mathematical correctness:

- **Rotation matrix properties**: Tests verify $R^T R = I$ (orthogonality) and $\det(R) = 1$ (orientation preservation) for all rotation matrices
- **Forward kinematics consistency**: Zero-angle configuration produces expected reach equal to sum of link lengths
- **IK convergence**: Error $\|\mathbf{p}_{target} - \mathbf{p}_{current}\|$ decreases monotonically for reachable targets
- **Singularity robustness**: Damping prevents NaN/infinity at fully extended configurations
- **Projection correctness**: Screen coordinates match manually computed values for known test cases
- **Jacobian numerical verification**: Finite difference approximation matches analytical computation

5 Linear Algebra Concepts Summary

The following table maps each linear algebra concept to its implementation module and specific application in the project:

Concept	Module	Application
Vector dot product	vectors.py	Angle computation, camera basis
Vector cross product	vectors.py	Jacobian columns, normal vectors
Matrix multiplication	transforms.py	FK chain, projection pipeline
Special Orthogonal Group SO(3)	transforms.py	Joint rotation matrices
Homogeneous coordinates	transforms.py	Unified rotation + translation
Jacobian matrix	inverse.py	Joint-to-Cartesian velocity mapping
Singular Value Decomposition	inverse.py	Stable damped pseudoinverse
Matrix inversion	transforms.py	Transform inverse ($R^{-1} = R^T$)
Orthonormal basis	camera.py	View matrix construction
Perspective projection	projection.py	Depth foreshortening effect

6 Expected Results and Design Validation

Based on the theoretical foundations and algorithmic design, the system is expected to demonstrate the following characteristics:

- **Convergence properties**: The DLS method should converge for reachable targets within 10-20 iterations based on literature [1], with convergence rate dependent on the damping factor $\lambda = 0.5$
- **Numerical stability**: SVD-based damped pseudoinverse is designed to handle singularities at fully extended arm configurations by thresholding small singular values
- **Performance targets**: Matrix operations in NumPy and Pygame rendering should enable interactive frame rates (target: 30-60 FPS) for the 3-DOF arm configuration
- **Workspace coverage**: The robot arm should reach any point within the spherical workspace defined by $r_{min} \leq \|\mathbf{p}_{target}\| \leq r_{max}$ where $r_{max} = \sum l_i$
- **Educational clarity**: The modular architecture with explicit linear algebra operations makes the mathematical concepts directly observable in code

6.1 Preliminary Validation

Unit tests have been designed to verify mathematical correctness of core components:

- Vector operations (dot/cross products) validated against known mathematical properties
- Rotation matrices verified to satisfy $R^T R = I$ and $\det(R) = 1$
- Homogeneous transformation composition tested for associativity
- Forward kinematics produces expected end-effector positions for known joint configurations

Full integration testing and performance benchmarking will be conducted in the final implementation phase to validate the convergence behavior, measure actual frame rates, and verify IK solver robustness across the full workspace.

7 Conclusion

This report presents the theoretical foundations and implementation design of an interactive 3D robot arm simulator demonstrating fundamental linear algebra concepts. The system architecture combines forward kinematics via matrix chain multiplication, inverse kinematics through Jacobian-based optimization with SVD-regularized pseudoinverses, and 3D rendering using perspective projection matrices.

The mathematical framework validates that linear algebra methods provide a robust, generalizable approach to inverse kinematics for articulated robot arms. The damped least squares algorithm with SVD-based pseudoinverse computation is designed to handle kinematic singularities while maintaining numerical stability. The visualization component uses homogeneous transformation matrices and camera view transformations constructed from orthonormal bases to project 3D arm configurations to 2D screen space.

The modular architecture—separating linear algebra primitives (vectors, transforms), kinematics computations (FK, IK), rendering (projection, camera), and visualization (interactive app)—enables clear mapping between mathematical theory and practical implementation. Each linear algebra concept (rotation groups, Jacobian matrices, SVD decomposition, perspective projection) is isolated in dedicated modules with comprehensive documentation.

The next phase involves completing the integration of all components, conducting comprehensive testing to validate convergence properties and numerical stability, and benchmarking performance to ensure real-time interactive operation. Future enhancements could include extending to 6-DOF spatial arms, implementing obstacle avoidance constraints, or exploring alternative IK methods (e.g., FABRIK) for comparative analysis.

Repository: `https://github.com/MunenNoKage/Robot_Arm_Inverse_Kinematic_Vis.git`

References

[1] S. Buss, "Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods," 2004. Available: `https://math.ucsd.edu/~sbuss/ResearchWeb/ikmethods/iksurvey.pdf`

[2] G. Strang, *Introduction to Linear Algebra*, 5th ed., Wellesley-Cambridge Press, 2016.

[3] R. Murray, Z. Li, S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.

[4] Scratchapixel, "The Perspective and Orthographic Projection Matrix." Available: `https://www.scratchapixel.com/lessons/3d-basic-rendering/perspective-and-orthographic-projection-matrix`